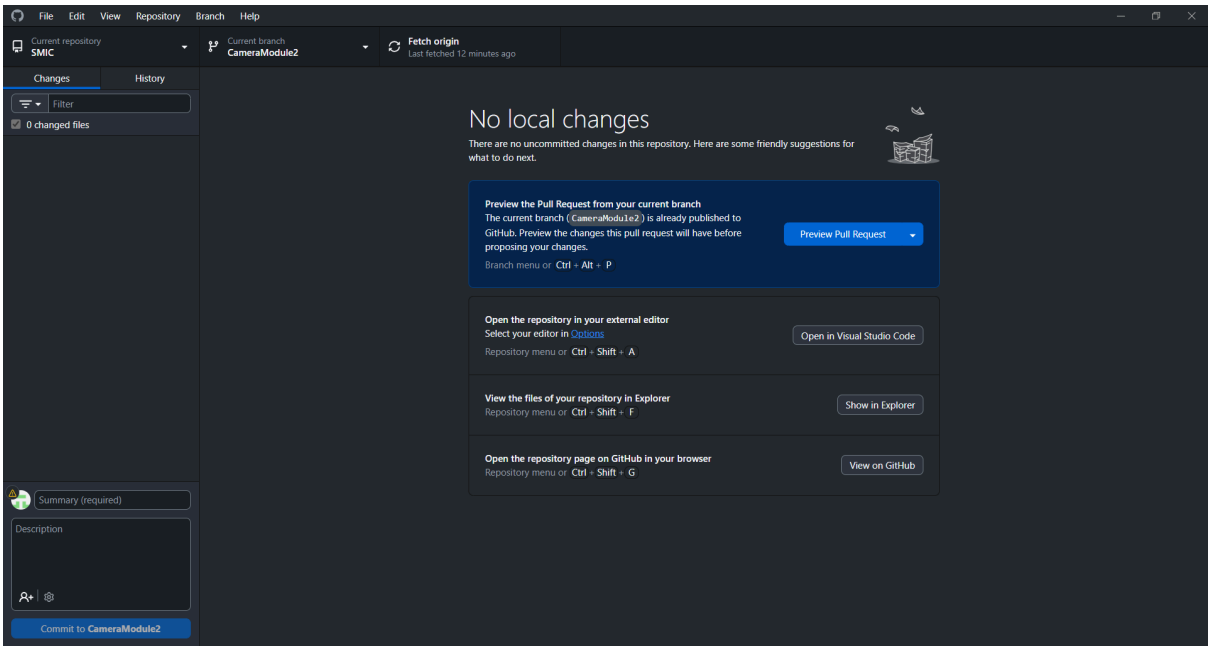
Camera Module 2

En el dia de hoy nos propusimos arrancar con el funcionamiento de la camara biometrica, conocer su funcionamiento, obtener librerias y llegar a probar un prototipo. Tambien crear un boceto de lo que seria la web donde de alojan los datos

Primero, creamos una rama desde main para poder trabajar sin ningun incoveniente, una vez creada, publicamos la ramay empezamos a trabajar desde aqui



Probaremos el sistema desde vs code.

Para trabajar correctamente, primero instalamos Python. Una vez instalado, verificamos que se haya instalado correctamente y que estemos en la version adecuada.

Una vez verificado lo anterior, abrimos la carpeta del proyecto en vs code. Creamos el entorno virtual dentro de la carpeta, el cual va a hacer que todas las librerias que instalemos para el proyecto queden almacenadas unicamente para el proyecto y si los integrantes de mi equipo quieren trabajar sobre mi archivo, lo van a poder trabajar sobre las versiones que utiliza el proyecto.

Instalamos las librerias que utilizaremos en el proyecto con el siguiente codigo

pip install opencv-python mediapipe flask flask-sqlalchemy requests bleak cloudinary

Que instalamos?

opencv-python es la biblioteca de visión artificial. Es la que va a analizar las imágenes de la cámara en tiempo real. En el SMIC la usás para abrir la cámara, leer cada frame, detectar

el rostro del conductor y dibujar los rectángulos de detección. Sin OpenCV no hay detección de somnolencia ni distracción.

mediapipe es de Google y va un paso más allá que OpenCV. Mientras OpenCV detecta que hay un rostro, MediaPipe mapea 468 puntos exactos de la cara, incluyendo los bordes de los párpados. Con esos puntos calculás el Eye Aspect Ratio (EAR), que es el número que determina si los ojos están cerrados. Es la biblioteca más importante para la detección de somnolencia del SMIC

flask es el servidor web del proyecto. Es el que recibe los datos que manda la Raspberry Pi y los guarda. También sirve las páginas HTML del panel web donde las empresas y aseguradoras van a ver los eventos del conductor. Sin Flask no hay plataforma web.

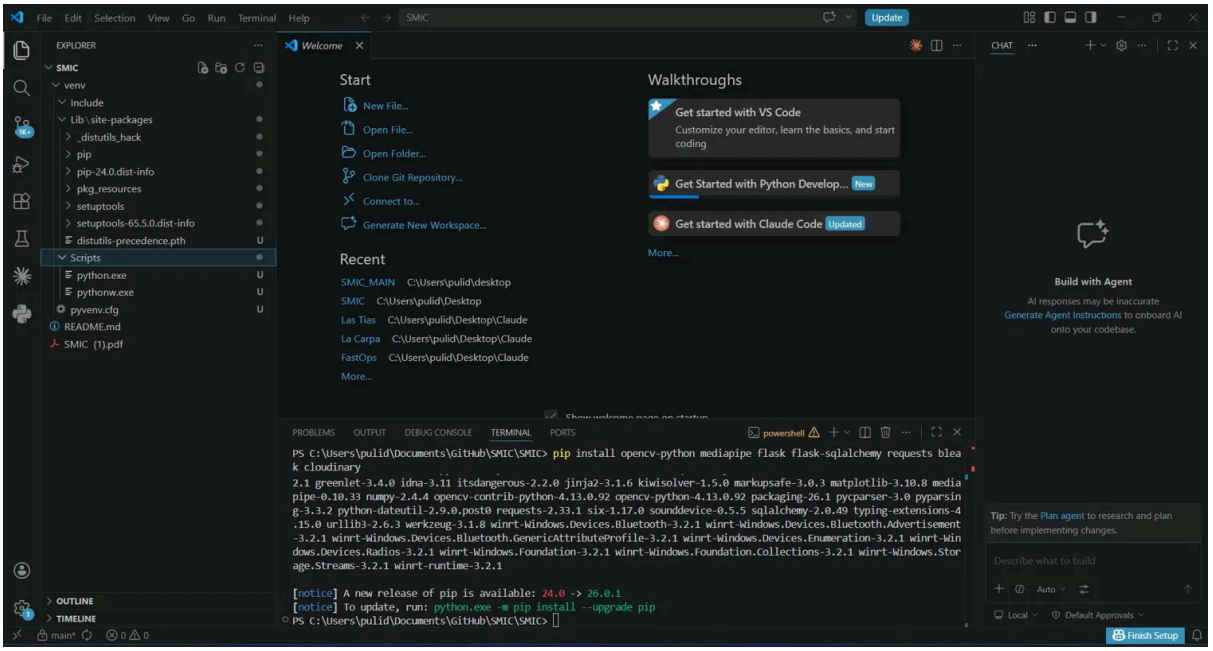
flask-sqlalchemy es el puente entre Flask y la base de datos. Te permite guardar y consultar eventos sin escribir SQL directamente. Vos definís el modelo en Python y él se encarga del resto. Por ejemplo, un `Evento` con campos como timestamp, bpm, temperatura y tipo de alerta.

requests sirve para hacer llamadas HTTP desde Python. La Raspberry Pi la usa para enviar los datos de los sensores a la API Flask. Es como el cartero del sistema: toma el JSON con los datos biométricos y lo manda al servidor.

bleak es la biblioteca Bluetooth Low Energy para Python. Es la que usa la Raspberry Pi para recibir los datos del ESP32 que está en el volante. Sin bleak la RPi no puede escuchar la señal BLE del ESP32 y los datos del MAX30102, DS18B20 y FSR nunca llegan.

cloudinary es el servicio de almacenamiento de videos en la nube. Cuando ocurre un evento, el clip de video se sube automáticamente a Cloudinary con una sola línea de código y te devuelve una URL. Esa URL se guarda en la base de datos y desde el panel web podés ver el video con un clic.

PROBLEMA



Las bibliotecas se instaron globalmente, no en el entorno virtual, ademas de que se instalaron versiones las cuales no necesitabamos.

Para solucionarlo tuvimos que ejecutar:

Activar el entorno virtual:

venv\Scripts\activate

Tambien solucionar el problema de execution policy:

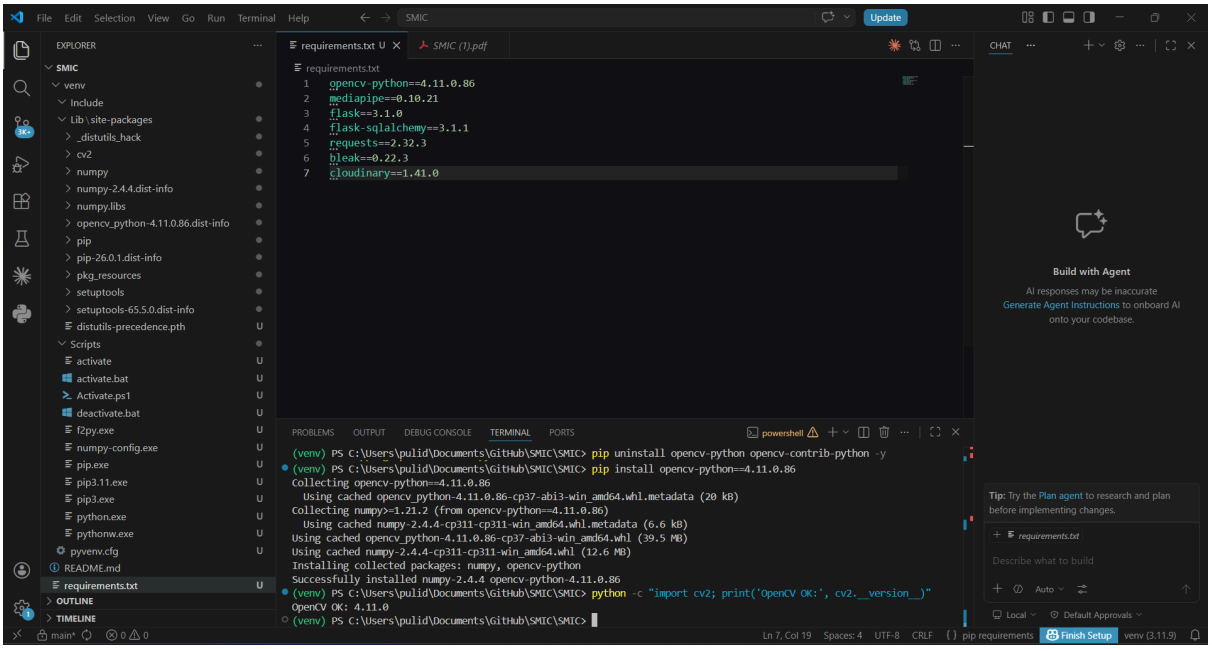
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Crear el requirements.txt para instalar las versiones deseadas, creamos el archivo y le pegamos la siguiente:

opencv-python==4.11.0.86
mediapipe==0.10.21
flask==3.1.0
flask-sqlalchemy==3.1.1
requests==2.32.3
bleak==0.22.3
cloudinary==1.41.0

Instalamos las versiones deseadas dentro del entorno virtual:

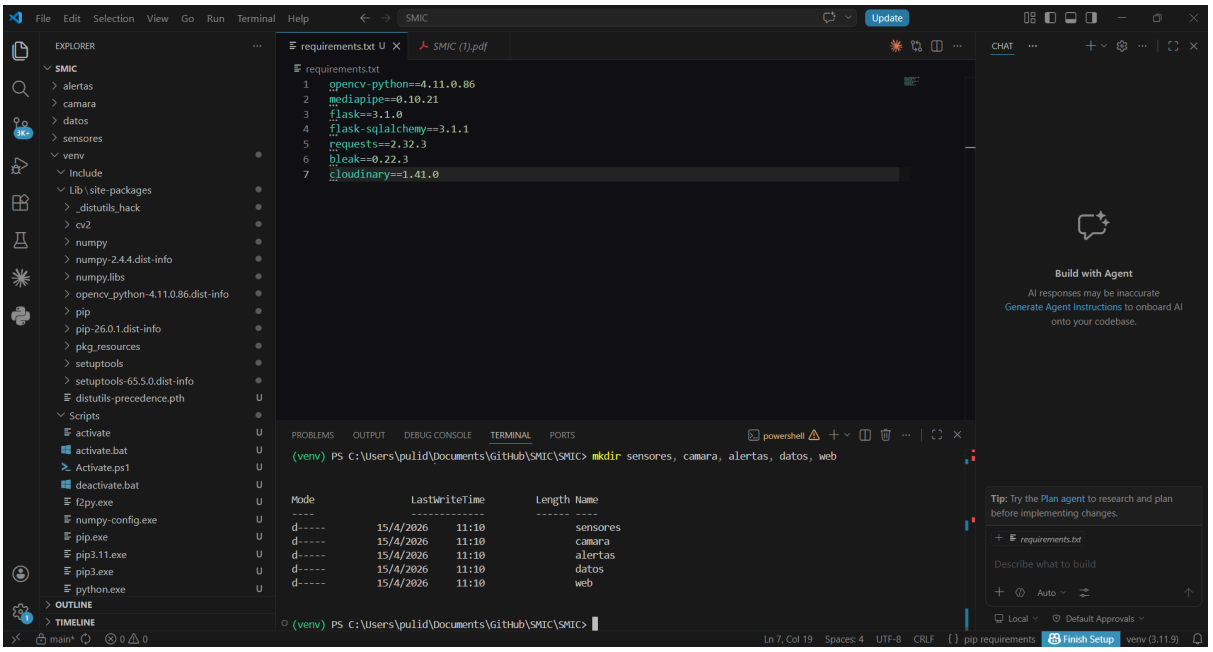
pip install -r requirements.txt



Con el entorno virtual creado y con las versiones correctas proseguimos.

Creamos las carpetas donde trabajaremos con las camaras, sensores, alertas, web, datos

mkdir sensores, camara, alertas, datos, web



Para empezar a trabajar debemos crear el archivo **__init__.py** el cual nos sirve para que python pueda reconocer la carpeta, ya que sin esta, si queremos importar un codigo de otra carpeta python dira que no existe. Nuestro proyecto necesita esto ya que, el main.py importara funciones de todas las carpetas, y sin el **__init__.py** no encontrara nada

Creamos los archivos **__init__.py** en cada carpeta para poder trabajar sin complicaciones en un futuro

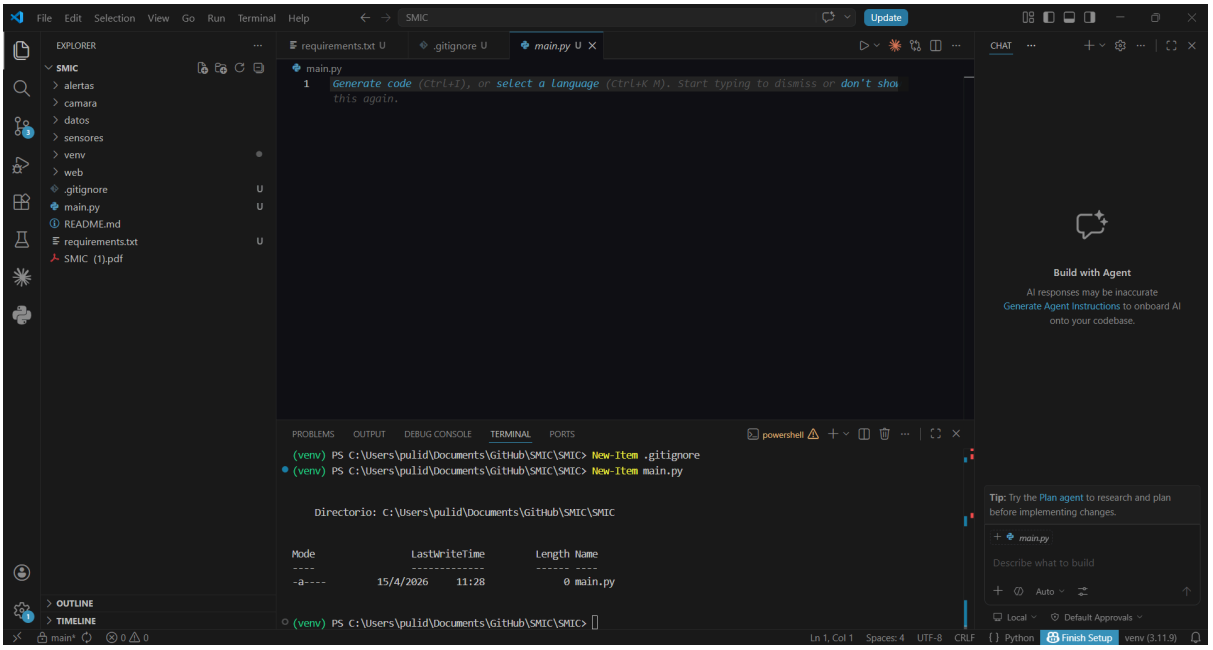
New-Item sensores__init__.py, camara__init__.py, alertas__init__.py, datos__init__.py, web__init__.py

Tambien creamos el .gitignore, el cual va a hacer que la carpeta venv(entorno virtual donde estan todas las librerias no se suban a git)

New-Item .gitignore

Creamos un archivo principal:

New-Item [main.py]



Todo correctamente instalado

Por ultimo, estableceremos el orden en el cual trabajaremos con los componentes y datos, el cual sera:

1. Sensores: Simularan los sensores del volante, no necesitamos el hardware ya mismo

2. Alertas: Simula el buzzer, tampoco necesita hardware

3. Camara: Deteccion facial con webcam de la notebook usando OpenCV y MediaPipe

4. Datos:Estructura de eventos en la base de datos

5. Web: Servidor Flask que recibe y muestra datos

6. Main: La union de todo